

让 ViT 跑得更快：我们试了三种方案，最终发现最简单的最有效

问题

ViT-B/16 处理一张 224×224 的图片时，会切成 **196 个 patch**，全部送入 12 层 transformer。这是 **33.85 GFLOPs** 的计算量。

但直觉告诉我们：不是所有 patch 都同等重要。背景、天空、重复纹理——这些 patch 对分类贡献很小。如果能在进入 transformer 之前**过滤掉不重要的 patch**，就能省下大量计算。

于是我们开始了探索。方向很明确，但路走了三条。

网络架构

在进入具体方案前，先理解模型的结构。

基线：全量 ViT-B/16

输入图片 (224×224)

↓

Patch Embedding (Conv2d, 16×16 , stride=16) → 196 patches, 每个 768 维

↓

Position Embedding + CLS Token

↓

12 层 Transformer Block (每层 = Attention + MLP + Residual)

↓

LayerNorm → 分类头 → 输出

Patch Selection 架构 (加了 Router)

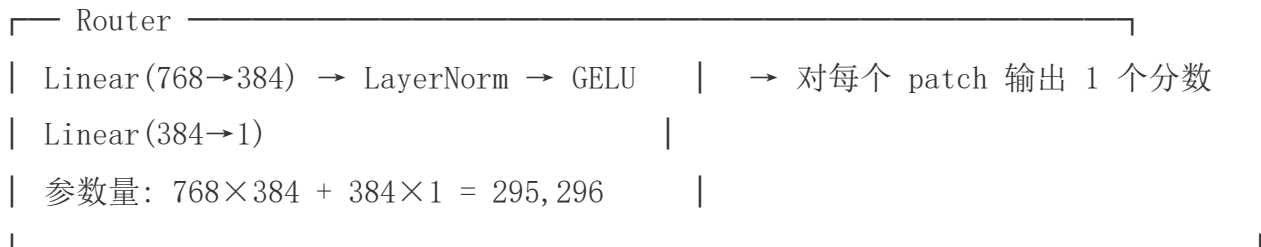
输入图片 (224×224)



Patch Embedding → 196 patches (768维)



Position Embedding (位置编码)



↓ 196 个分数

Differentiable Top-K 选择 (Sigmoid STE)

↓ 保留 K 个最重要的 patch

concat(CLS Token, 选中 patches) → (K+1, 768)



前 2 层 Transformer Block (轻量编码器)

↓ 保存中间特征 → 送入 MAE Decoder (训练时)

后 10 层 Transformer Block (主干网络)



LayerNorm → 分类头 → 输出

Router 的参数量 (296K) 仅为 ViT-B/16 整体 (85.8M) 的 0.34%，极小。

Differentiable Top-K 原理

选择"选 K 个 patch"是离散操作，没法直接算梯度。用 STE 技巧绕过：

```
scores = router(patch_features)          # 196 个分数
threshold = scores.topk(K)[-1:]          # 第 K 大的分数作为阈值
soft_mask = sigmoid(scores - threshold)  # 连续的可微分掩码
hard_mask = (scores > threshold).float() # 离散的选/不选
# STE: 前向用 hard_mask, 反向梯度直穿 soft_mask
mask = hard_mask.detach() + soft_mask - soft_mask.detach()
```

基线：全量 ViT-B/16 微调

所有对比的基准是 ViT-B/16 (85.8M 参数) 在 ImageNet-21K 上预训练后，在目标数据集上全量微调。

训练配置：

- 优化器：AdamW, lr=3e-5, weight_decay=0.05
- 调度器：CosineAnnealing, 100 epochs (CIFAR-100) / 30 epochs (Food-101)
- 数据增强：RandomResizedCrop + RandomHorizontalFlip
- 标签平滑：0.1
- 有效 batch size：128 (batch=32 × accum=4)

数据集	类别	原生分辨率	Test Acc	适合降采样实验
Oxford Pets	37	~200-1000px (高)	93.81%	✅ 主数据集
Food-101	101	~300-512px (高)	91.37%	✅
DTD	47	~300-400px (高)	80.85%	✅ 纹理敏感
CIFAR-100	100	32x32 (极低)	91.69%	❌ 不适合降采样

关于 CIFAR-100：原图仅 32x32，训练时放大到 224x224。本身就没多少细节，降采样测试会给出"几乎不掉精度"的假象。CIFAR-100 仅适合测试 patch 级别的选择策略（如 Router），不适合测试降采样/分辨率相关的实验。所有降采样实验应以 Oxford Pets、Food-101 等高分辨率数据集为准。

第一条路：Gumbel Selection —— 学不动

思路：加一个小型 MLP Router (~300K 参数) 给每个 patch 打分，用 Gumbel-Softmax + STE 做可微分的"选/不选"，只把选中的 patch 送进 backbone。

结果 (CIFAR-100, 保留 50%)：

方法	Acc	vs 全量
全量 ViT-B/16	91.69%	—

Gumbel Selection	87.18%	-4.51%
------------------	--------	--------

问题在哪？ Gumbel 噪声太大了。Router 想表达"这个 patch 重要，score=0.8"，但噪声一加，变成了随机数。梯度信号被淹没，Router 学了一个周期，最后还是没学会。

教训： 用噪声做离散化选择，噪声强度控制不好就白学。

第二条路：MAE + 蒸馏 Router —— 有效，但复杂

思路： 去掉 Gumbel，改用 Sigmoid STE（梯度直通），让梯度直接穿过"选/不选"的门槛。同时加一个 **MAE 重建任务**：Encoder 处理后，用小 Decoder 重建被丢弃的 patch，用 MSE Loss 告诉 Router"你丢的东西能不能猜回来"。

这相当于给 Router 配了两个教练：

- **分类 Loss** 告诉它：选出来的 patch 要能分类
- **重建 Loss** 告诉它：丢掉的 patch 不能是关键信息

方法	保留	Acc	vs 全量
全量 ViT-B/16	100%	91.69%	—
MAE + 随机 Router	50%	88.25%	-3.44%
MAE + 蒸馏 Router	50%	89.07%	-2.62%
MAE + 蒸馏 Router	75%	91.10%	-0.59%

保留 75% 时吞吐量提升 27%，保留 50% 时提升 73%。

但有个细节： 蒸馏 Router 的注意力重叠率只有 55.76%（随机基线 50%），几乎没学到。核心收益来自 MAE 而非蒸馏。

教训： 学习型方案能工作，但训练流程复杂（两阶段），且核心收益来自 MAE。

第三条路：降采样训练 —— 简单到让人怀疑

"如果我只是把输入图片缩小一点，会怎么样？"

原图 224×224 → Resize(168×168) → Patch Embed → 100 patches → ViT → 分类

不需要 Router、不需要蒸馏、不需要 MAE。就一行 Resize。

但要小心：CIFAR-100 是个陷阱

CIFAR-100 原图只有 32×32，放大到 224×224 训练本来就没细节。在 CIFAR-100 上测降采样得到"几乎不掉精度"的结论是假的——信息根本没有损失。

换到真正的高分辨率数据集上测试，结果完全不同：

数据集	特点	Baseline	168×168	下降
CIFAR-100	粗粒度，原生 32×32	91.69%	91.56%	-0.13% (假象)
Food-101	细粒度菜品	91.37%	89.87%	-1.50%
Oxford Pets	猫狗品种，花纹重要	93.81%	90.65%	-3.16%
DTD	纯纹理分类	80.85%	74.20%	-6.65%

精度下降幅度与数据集的信息依赖类型高度相关：粗粒度物体几乎不掉 → 细粒度掉 1.5-3% → 纯纹理掉最多（6.65%）。降采样有效与否，取决于任务所需的信息频段。

效率提升 (Oxford Pets)：

输入尺寸	Patches	Acc	吞吐量
224×224	196	93.81%	791/s
168×168	100	90.65%	1458/s (+84%)
112×112	49	86.64%	2820/s (+257%)

隔行去行列（直接丢掉一半像素）比双线性降采样略差（低 0.5%），平滑插值保留更多信息。

灰度图实验（颜色信息剥离）

去掉颜色信息只保留亮度，ViT 还能分类吗？

数据集	Baseline (RGB)	Grayscale	下降
Oxford Pets	93.81%	90.68%	-3.13%

去掉颜色掉 3.13%，说明 Oxford Pets 上颜色信息有一定作用（部分品种靠毛色区分），但形状和纹理更重要。

但有个坑：不是所有任务都这么宽容

数据集	全量	168×168	下降幅度
CIFAR-100	91.69%	91.56%	-0.13%
Food-101	91.37%	89.87%	-1.50%

Food-101 掉点多了 10 倍。CIFAR-100 的物体靠轮廓区分，Food-101 的食物靠纹理区分。细粒度分类对分辨率更敏感。

分类粒度越细，对分辨率越敏感。粗粒度适合降采样，细粒度要谨慎。

如果推理时缩小（不重新训练）呢？

效果很差：

- 168×168 推理硬切：89.24% (vs 重新训练 91.56%)
- 112×112 推理硬切：73.71% (vs 重新训练 90.00%)

ViT 的位置编码在训练时学会的，推理时改变输入尺寸对齐全乱。**必须重新训练。**

总结：跨数据集对比

数据集	特点	Baseline	168×168	下降	112×112	下降
CIFAR-100	粗粒度, 32×32原生	91.69%	91.56%	-0.13%	90.00%	-1.69%
Food-101	细粒度菜肴	91.37%	89.87%	-1.50%	85.96%	-5.41%
Oxford Pets	猫狗品种, 花纹纹理	~93.3%	90.65%	-3.16%	86.64%	-7.17%
DTD	纯纹理分类	80.85%	74.20%	-6.65%	—	—

核心结论

- 降采样伤害取决于任务所需的信息类型**——粗粒度几乎不受影响，细粒度掉 1.5-3%，纹理分类掉最多（6.65%）。越依赖高频细节的任务，降采样伤害越大。
- 颜色信息有一定作用但不关键**——Oxford Pets 灰度图掉 3.13%，部分品种靠毛色区分。
- 学习型方案 vs 降采样**——待跑完随机/蒸馏 Router 对比后再下结论。

欢迎讨论和反馈。